LLM Behaviour Distillation in Open-Source Game Theory: Implementation and Contributions

Dominic Satnoianu
Collaborating on EuroSafeAI's Open-Source Game Theory Project

Current implementation

Before my contributions, the main branch of the project repository already contained a self-contained Python package that distils an LLM's behaviour into the open-source game theory (OSGT) setting of Critch et al. [3] and Barasz et al. [2].

The model plays a one-shot Prisoner's Dilemma under full transparency. It is given a prompt explaining the rules of the game together with the source code of its opponent (written as Python-like pseudocode rather than the Lean source), and it replies with an action, Cooperate (`C`) or Defect (`D`), and a short explanation of its reasoning. This is repeated against each bot in a fixed library (the *zoo*) of nine well-studied strategies, from CooperateBot and DefectBot to the proof-based CupodBot and DupocBot. By default, each bot is queried 30 times at temperature 1.0 via OpenRouter. The result is the model's *cooperation profile* $x \in [0,1]^9$, where x_i is the fraction of valid replies in which the model cooperates against the i-th bot.

This profile is then fitted against the zoo: we look for non-negative weights summing to 1 such that the corresponding mixture of the bots' own behaviour comes closest to x. Closeness is measured in several ways: the L_1 and L_∞ distances (solved as linear programs), the L_2 distance (a quadratic program), and an expected-Hamming baseline, which reduces to picking the single nearest bot. A nonzero residual means that no mixture of library bots reproduces the model's profile.

My contributions are listed below. They may change as I discuss them further with the team. All of them live in my fork of the repository.

Contribution: Re-fitting utility, sample runs and testing

Querying a model takes about 270 prompts (9 bots $\times$ 30 samples), which is roughly 15 minutes on a model such as Claude Haiku 5.5. To avoid repeating this whenever the fitting step changes, I wrote a re-fitting utility, `refit_run.py`, which recomputes every fit from the raw replies saved by an earlier run, without calling the model again.

I also committed the first sample runs in the repository, covering models from different families (`deepseek/deepseek-v4.1-flash`, `recursive/flint-v1.0:free` and the `openrouter/free` router) at temperatures 0.0 and 1.0. These runs use only 3 to 5 samples per bot, so they serve as a test of the pipeline rather than as results, but there could definitely be more experiments like this published.

Finally, I fixed a unit test that had gone out of date. The test asserted that CupodBot and DupocBot behave identically against every bot in the zoo, which stopped being true once CupodTrollBot was added to the payoff matrix. The new test checks that the two bots differ *only* against CupodTrollBot. I also added a GitHub Actions workflow that runs the test suite whenever the distillation code changes.

Contribution: Binomial fitting method

The existing fits work only with the profile x, so a bot against which the model gave 3 valid replies counts as much as one with 30, provided the two cooperation rates are equal. To account for this, I implemented `fit_binomial`, which fits the bot mixture by binomial maximum likelihood on the raw counts. If c_j of the n_j valid replies against bot j are `C`, and $p = R^\top w$ is the cooperation profile of the mixture with weights w, it minimises

$$-\sum_j \Big[\, c_j \log p_j + (n_j - c_j) \log(1 - p_j) \,\Big]$$

over the probability simplex. The reported residual is the deviance $2\big(\mathrm{NLL}(p) - \mathrm{NLL}(x)\big)$, which is zero exactly when x is a mixture of library bots. It is on a different scale from the L_p residuals, so residuals should only be compared within the same metric.

Ongoing work and next steps

This list is not exhaustive, subject to change as I further discuss with my team, and decide what I can disclose or not.

- I have found two recent papers that learn formal models from LLM behaviour, and I plan to explore how either can be adapted to our setting. They both build on Angluin's L^* algorithm [1] for automata learning, and primarily learn from sequences of inputs such as move histories in an iterated Prisoner's Dilemma setting, whereas OSGT is a one-shot game.
- Exploring alternative fitting strategies.
- A richer and more comprehensive zoo would help mitigate some results such as measured profiles lying outside the hull. This would primarily be done using further bots from literature. All three models defect against CooperateBot, DBot, DefectBot, EBot and CupodTrollBot, cooperate against CupodBot, almost always cooperate against DupocBot, and are mixed against OBot and TitForTatBot. Every fitting method leaves a large residual, so this profile lies well outside the hull of the zoo. With so few samples, it would be ideal to test this hypothesis or pattern out.

- A more exhaustive set of runs, with more samples per bot and more models at different configurations, would also help confirm this.

References

[1] D. Angluin. Learning regular sets from queries and counterexamples. *Information and Computation*, 75(2):87–106, 1987.

[2] M. Barasz, P. Christiano, B. Fallenstein, M. Herreshoff, P. LaVictoire and E. Yudkowsky. Robust cooperation in the Prisoner's Dilemma: Program equilibrium via provability logic. arXiv:1401.5577, 2014.

[3] A. Critch, M. Dennis and S. Russell. Cooperative and uncooperative institution designs: Surprises and problems in open-source game theory. arXiv:2208.07006, 2022.

[4] I. D. Lopez-Miguel, A. Happe, J. Cito, E. Bartocci, B. Könighofer and M. Tappler. ATLAS: Discovering agent strategies through LLM-guided abstraction and automata learning. In *MODELS*, 2026. arXiv:2608.14352.

[5] M. Vazquez-Chanlatte, K. Elmaaroufi, S. Witwicki and S. A. Seshia. L^*LM: Learning automata from examples using natural language oracles. arXiv:2402.07051, 2024.

[6] G. Weiss, Y. Goldberg and E. Yahav. Extracting automata from recurrent neural networks using queries and counterexamples. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018.